Status	Finished
Started	Wednesday, 21 October 2026, 12:08 PM
Completed	Wednesday, 21 October 2026, 12:14 PM
Duration	6 mins 22 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Determine the factors of a number (i.e., all positive integer values that evenly divide into a number) and then return the pth element of the list, sorted ascending. If there is no p^{th} element, return 0.

Example

n = 20

p = 3

The factors of 20 in ascending order are {1, 2, 4, 5, 10, 20}. Using 1-based indexing, if p = 3, then 4 is returned. If p > 6, 0 would be returned.

Constraints

$1 \leq n \leq 10^{15}$

$1 \leq p \leq 10^{9}$

The first line contains an integer n, the number to factor.

The second line contains an integer p, the 1-based index of the factor to return.

Sample Case 0

Sample Input 0

10

3

Sample Output 0

5

Explanation 0

Factoring n = 10 results in {1, 2, 5, 10}. Return the p = 3rd factor, 5, as the answer.

Sample Case 1

Sample Input 1

10

5

Sample Output 1

0

Explanation 1

Factoring n = 10 results in {1, 2, 5, 10}. There are only 4 factors and p = 5, therefore 0 is returned as the answer.

Sample Case 2

Sample Input 2

1

1

Sample Output 2

1

Explanation 2

Factoring n = 1 results in {1}. The p = 1st factor of 1 is returned as the answer.

For example:

Input	Result
10 3	5
10 5	0

Input	Result
1 1	1

Answer: (penalty regime: 0 %)

```
import math

n = int(input())
p = int(input())

small = []
large = []

for i in range(1, math.isqrt(n) + 1):
    if n % i == 0:
        small.append(i)

        if i != n // i:
            large.append(n // i)

factors = small + large[::-1]
```

	Input	Expected	Got	
✓	10 3	5	5	✓
✓	10 5	0	0	✓
✓	1 1	1	1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given a matrix mat where every row is sorted in **strictly increasing** order, return the **smallest common element** in all rows.

If there is no common element, return -1.

Example 1:

Input:

4 5

1 2 3 4 5

2 4 5 8 10

3 5 7 9 11

1 3 5 7 9

Output:

5

Constraints:

- 1 <= mat.length, mat[i].length <= 500
- 1 <= mat[i][j] <= 10^4
- mat[i] is sorted in strictly increasing order.

Answer: (penalty regime: 0 %)

```
m, n = map(int, input().split())

mat = []
for i in range(m):
    row = list(map(int, input().split()))
    mat.append(row)

common = set(mat[0])

for i in range(1, m):
    common = common.intersection(set(mat[i]))

if len(common) == 0:
    print(-1)
else:
    print(min(common))
```

	Input	Expected	Got	
✓	4 5 1 2 3 4 5 2 4 5 8 10 3 5 7 9 11 1 3 5 7 9	5	5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given two arrays of positive integers, for each element in the second array, find the total number of elements in the first array which are *less than or equal to* that element. Store the values determined in an array.

For example, if the first array is *[1, 2, 3]* and the second array is *[2, 4]*, then there are *2* elements in the first array *less than or equal to 2*. There are *3* elements in the first array which are *less than or equal to 4*. We can store these answers in an array, *answer = [2, 3]*.

Program Description

The program must return an array of *m* positive integers, one *for each maxes[i]* representing the total number of elements *nums[j]* satisfying *nums[j] ≤ maxes[i]* where *0 ≤ j < n* and *0 ≤ i < m,* in the given order.

The program has the following:

nums[nums[0],...nums[n-1]]: first array of positive integers
maxes[maxes[0],...maxes[n-1]]: second array of positive integers

Constraints

- *2 ≤ n, m ≤ 10^5*
- *1 ≤ nums[j] ≤ 10^9*, where *0 ≤ j < n*.
- *1 ≤ maxes[i] ≤ 10^9*, where *0 ≤ i < m*.

Input Format For Custom Testing

Input from stdin will be processed as follows and passed to the program.

The first line contains an integer *n*, the number of elements in *nums*.
The next *n* lines each contain an integer describing *nums[j]* where *0 ≤ j < n.*
The next line contains an integer *m*, the number of elements in *maxes*.
The next *m* lines each contain an integer describing *maxes[i]* where *0 ≤ i < m.*

Sample Case 0

Sample Input 0

```
4
1
4
2
4
2
3
5
```

Sample Output 0

```
2
4
```

Explanation 0

We are given *n = 4*, *nums = [1, 4, 2, 4]*, *m = 2*, and *maxes = [3, 5]*.

1. For *maxes[0] = 3*, we have *2* elements in *nums* (*nums[0] = 1* and *nums[2] = 2*) that are *≤ maxes[0]*.
2. For *maxes[1] = 5*, we have *4* elements in *nums* (*nums[0] = 1*, *nums[1] = 4*, *nums[2] = 2*, and *nums[3] = 4*) that are *≤ maxes[1]*.

Thus, the program returns the array *[2, 4]* as the answer.

Sample Case 1

Sample Input 1

```
5
2
10
5
4
8
4
3
1
7
8
```

Sample Output 1

```
1
0
3
4
```

Explanation 1

We are given, *n = 5*, *nums = [2, 10, 5, 4, 8]*, *m = 4*, and *maxes = [3, 1, 7, 8]*.

1. For *maxes[0] = 3*, we have *1* element in *nums* (*nums[0] = 2*) that is *≤ maxes[0]*.
2. For *maxes[1] = 1*, there are *0* elements in *nums* that are *≤ maxes[1]*.
3. For *maxes[2] = 7*, we have *3* elements in *nums* (*nums[0] = 2*, *nums[2] = 5*, and *nums[3] = 4*) that are *≤ maxes[2]*.
4. For *maxes[3] = 8*, we have *4* elements in *nums* (*nums[0] = 2*, *nums[2] = 5*, *nums[3] = 4*, and *nums[4] = 8*) that are *≤ maxes[3]*.

Thus, the program returns the array *[1, 0, 3, 4]* as the answer.

Answer: (penalty regime: 0 %)

```
n = int(input())
nums = []

for i in range(n):
    nums.append(int(input()))

m = int(input())
maxes = []

for i in range(m):
    maxes.append(int(input()))

nums.sort()

for x in maxes:
    count = 0
    for num in nums:
```

	Input	Expected	Got	
✓	4 1 4 2 4 2 3 5	2 4	2 4	✓
✓	5 2 10 5 4 8 4 3 1 7 8	1 0 3 4	1 0 3 4	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct